

Swarm Invasion

Dokumentacja wstępna

26 maja 2026

1. Wstęp

Celem projektu **Swarm Invasion** jest stworzenie dwuosobowej lub wieloosobowej gry akcji typu *top-down horde shooter*, w której gracze łączą się z serwerem, wybierają klasę postaci i wspólnie odpierają kolejne fale przeciwników. Główny nacisk w projekcie położono na architekturę klient-serwer, podstawowe systemy rozgrywki, synchronizację stanu świata oraz możliwość dalszego rozwijania klas, broni i zachowań AI.

Program składa się z dwóch wykonywalnych części: klienta odpowiedzialnego za wejście użytkownika, renderowanie i interfejs oraz serwera odpowiedzialnego za obsługę połączeń sieciowych, aktualizację przeciwników, postęp fali i autorytatywny stan świata. Obie części współdzielą warstwę definicji protokołu, konfiguracji i generatora mapy. Parametry balansu rozgrywki przechowywane są w zewnętrznych plikach JSON.

Niniejsza dokumentacja przedstawia zastosowane rozwiązania architektoniczne, zaimplementowane mechaniki rozgrywki oraz strukturę kodu na obecnym etapie rozwoju silnika.

2. Architektura programu

2.1. Podział na warstwy

Projekt jest podzielony na trzy główne części:

- **klient** (`src/client`) – obsługuje okno SFML, UI w ImGui, wejście użytkownika, lokalną reprezentację gracza, renderowanie mapy, obiektów i przeciwników oraz część logiki pocisków,
- **serwer** (`src/server`) – utrzymuje listę połączonych graczy, stan przeciwników, fale, leczenie, doświadczenie drużyny i okresowo wysyła stan świata,
- **warstwa współdzielona** (`src/shared`) – zawiera definicje pakietów sieciowych, stałe konfiguracyjne, rejestry danych wczytywanych z JSON oraz generator mapy.

2.2. Punkty wejścia

Uruchomienie klienta rozpoczyna się w pliku `src/client/core/main.cpp`. Program wczytuje konfiguracje broni, bohaterów i przeciwników z katalogu `assets`, tworzy obiekt `ClientEngine` i uruchamia główną pętlę klienta.

Uruchomienie serwera realizowane jest przez `src/server/core/main.cpp`. Podobnie jak klient, serwer najpierw wczytuje konfigurację z plików JSON, następnie tworzy `ServerEngine` i przechodzi do pętli aktualizacji z krokiem równym 1/60 sekundy.

2.3. Budowanie projektu

Projekt budowany jest przy pomocy CMake. W pliku `CMakeLists.txt` zdefiniowano standard C++20 i pobieranie bibliotek przez `FetchContent`. Tworzone są dwa programy:

- `SwarmServer`,
- `SwarmClient`.

Użyte biblioteki zewnętrzne:

- SFML 3.1.0,
- ImGui 1.91.5,
- ImGui-SFML 3.0,
- `nlohmann/json` 3.12.0.

Podczas konfiguracji budowy katalog `assets` kopiowany jest do katalogu `build`, dlatego programy uruchamiane z katalogu budowania korzystają z zasobów znajdujących się właśnie tam.

3. Jednostki kompilacji i moduły

3.1. Część klienta

Najważniejsze moduły klienta są następujące:

- `ClientEngine` w pliku `src/client/core/ClientEngine.cpp` – główna pętla klienta, obsługa okna, zdarzeń, pakietów sieciowych oraz przełączania stanów,
- `LobbyState` w `src/client/states/LobbyState.cpp` – ekran wyboru adresu serwera i klasy postaci, wysyłający `JoinRequest`,
- `GameState` w `src/client/states/GameState.cpp` – główny stan rozgrywki, odpowiedzialny za wejście, kamerę, lokalną symulację pocisków, odbiór stanu świata, UI oraz powrót do lobby po śmierci,
- `MapRenderer` w `src/client/core/MapRenderer.cpp` – przygotowuje sposób wyświetlania całej mapy, tak aby mogła być sprawnie rysowana na ekranie,
- klasy entity w `src/client/entities` – bazą jest `Entity`, a `Player` i `Enemy` są klasami pochodnymi reprezentującymi odpowiednio graczy i przeciwników,
- system pocisków w `src/client/projectiles` – lokalna reprezentacja pocisku, menedżer pocisków i podstawowe kolizje.

Klient działa w oparciu o maszynę stanów: po utworzeniu `ClientEngine` aktywowany jest `LobbyState`, a po dołączeniu do gry następuje przejście do `GameState`.

3.2. Część serwera

Najważniejsze elementy serwera to:

- `ServerEngine` w `src/server/core/ServerEngine.cpp` – obsługa połączeń UDP, klientów, trafień, doświadczenia drużyny, pól leczenia, stanu świata i stanu rozgrywki,
- `AIDirector` w `src/server/core/AIDirector.cpp` – generowanie fal, wybór typu przeciwnika, flow-field do nawigacji oraz lokalna siatka przestrzenna przeciwników,
- klasy przeciwników w `src/server/enemies` – wspólna baza `ServerEnemy` oraz poszczególne typy wrogów: `ServerCrawler`, `ServerBruiser`, `ServerSpitter` i `ServerKamikaze`,
- struktury pomocnicze w `src/server/core/ServerTypes.hpp` – opis klienta, punktów doświadczenia i pola leczenia,
- `SpatialGrid` w `src/server/core/SpatialGrid.hpp` – uproszczony podział przestrzeni używany przez AI do lokalnego wyszukiwania sąsiadów.

Serwer działa jako *headless server*, czyli program uruchamiany bez interfejsu graficznego. Jego zadaniem jest aktualizacja świata i rozsyłanie najważniejszych informacji do klientów.

3.3. Warstwa współdzielona

Współdzielona część projektu zawiera:

- `NetworkProtocol.hpp` – definicje typów pakietów, klas graczy, typów przeciwników i typów broni, a także serializację podstawowych typów do `sf::Packet`,
- `Config.hpp` – stałe takie jak rozmiar okna, port serwera, rozmiar mapy, promień kolizji i parametry systemu XP,
- `HeroRegistry.hpp`, `WeaponRegistry.hpp`, `EnemyRegistry.hpp` – rejestry statystyk wczytywanych z plików JSON,
- `MapGenerator.cpp` – generator mapy oparty na prostym automacie komórkowym z wygładzaniem i centralną strefą startową.

4. Model rozgrywki

4.1. Bohaterowie

Docelowo projekt zakłada cztery klasy bohaterów, natomiast obecnie zaimplementowane są dwie: `Soldier` i `Juggernaut`. W planach dalszego rozwoju znajdują się kolejne klasy, między innymi `Vanguard` oraz `Engineer`.

Ich podstawowe parametry, takie jak maksymalne punkty życia, szybkość poruszania, promień i domyślna broń, przechowywane są w pliku `assets/heroes.json`. Obecny stan projektu koncentruje się przede wszystkim na rozwijaniu i testowaniu dwóch dostępnych klas.

4.2. Umiejętności bohaterów

Aktualnie zaimplementowane klasy różnią się nie tylko statystykami, ale również dostępnym zestawem akcji specjalnych. **Soldier** pełni rolę bardziej uniwersalnej klasy dystansowej: korzysta z podstawowego ostrzału karabinem, posiada sprint oparty na pasku staminy, alternatywny atak rakietowy, umiejętność tworzenia pola leczenia oraz ultimate, dający mu zwiększoną szybkostrzelność, nieskończoną amunicję oraz auto-namierzanie przeciwników na pewien okres czasu.

Juggernaut jest klasą cięższą i wolniejszą, opartą na walce z bliskiego dystansu. W obecnym stanie projektu korzysta z broni typu shotgun oraz z krótkiej szarży do przodu. Pozostałe sloty umiejętności tej klasy są już przewidziane w strukturze kodu i interfejsie, ale ich działanie jest jeszcze rozwijane.

4.3. Broń i pociski

Statystyki broni zapisane są w `assets/weapons.json`. Rejestr broni wczytuje m.in. prędkość pocisku, promień, obrażenia, czas życia pocisku, liczbę śrucin i kąt rozrzutu. Dzięki temu część balansu może być modyfikowana bez przebudowy kodu.

Po stronie klienta pociski aktualizowane są lokalnie przez `ProjectileManager`. Kolizja z przeciwnikiem lub graczem skutkuje utworzeniem rekordu trafienia, który następnie wysyłany jest do serwera jako pakiet `EntityHit`.

4.4. Przeciwnicy i fale

W grze występują cztery typy przeciwników:

- **Crawler** – podstawowy przeciwnik walczący wręcz,
- **Bruiser** – wolniejszy i wytrzymalszy przeciwnik z szarżą,
- **Spitter** – przeciwnik dystansowy strzelający kwasem,
- **Kamikaze** – szybki przeciwnik eksplodujący po dotarciu do celu.

Fale obsługiwane są przez `AIDirector`. Co pewien czas zwiększany jest numer fali, a odstęp pomiędzy spawnami przeciwników jest skracany. Pozycje przeciwników losowane są na mapie, lecz odrzucane, jeśli znajdują się zbyt blisko gracza lub w ścianie.

4.5. Mapa i kolizje

Mapa generowana jest proceduralnie po stronie klienta i serwera, zawsze z tym samym seedem po obu stronach. Generator tworzy dwuwymiarową siatkę kafli typu podłoga/ściana, wykonuje kilka kroków wygładzania i czyści obszar wokół środka mapy, aby zapewnić bezpieczne miejsce startu. Dzięki temu obie strony symulacji odtwarzają identyczną geometrię bez potrzeby przesyłania całej mapy przez sieć.

4.6. Doświadczenie i rozwój drużyny

Po śmierci przeciwników pojawiają się punkty doświadczenia. Po zebraniu odpowiedniej liczby punktów poziom drużyny wzrasta, gra zostaje chwilowo zatrzymana, a klient wyświetla okno wyboru ulepszenia. Obecna implementacja obejmuje UI wyboru i obsługę timeoutu po stronie serwera, ale rzeczywiste efekty kart nie są jeszcze zaimplementowane.

5. Komunikacja sieciowa

5.1. Model połączenia

Komunikacja sieciowa opiera się na bezpołączeniowych gniazdach UDP z biblioteki SFML. Zarówno klient, jak i serwer pracują w trybie nieblokującym. Serwer nasłuchuje na porcie 54000, a klient regularnie wysyła swoją pozycję oraz pakiety akcji.

5.2. Najważniejsze pakiety

Warstwa protokołu definiuje m.in. następujące typy pakietów:

- JoinRequest, JoinAccept,
- PlayerPosition,
- PlayerShoots,
- EntityHit,
- WorldState,
- LevelUpTriggered,
- SpawnHealField,
- PlayerDealtDamage,
- PlayerDisconnect.

5.3. Stan świata

Serwer okresowo wysyła pakiet `WorldState`, zawierający:

- listę aktywnych graczy i ich pozycje oraz HP,
- poziom drużyny i stan paska doświadczenia,
- listę przeciwników i ich pozycje oraz HP,
- pozycje punktów doświadczenia.

Klient wykorzystuje ten stan świata do synchronizacji położenia innych graczy i przeciwników. Lokalny gracz nadal porusza się jednak po stronie klienta na podstawie wejścia użytkownika.

5.4. Autorytatywność i ograniczenia modelu

Model synchronizacji jest hybrydowy:

- serwer jest autorytatywny dla przeciwników, HP, fal, doświadczenia i pól leczenia,
- klient raportuje własną pozycję i zgłasza trafienia pociskami.

Oznacza to, że część informacji o działaniach gracza trafia do serwera z poziomu klienta, natomiast kluczowy stan rozgrywki pozostaje utrzymywany centralnie po stronie serwera. Mimo że UDP nie zapewnia niezawodności ani kolejności dostarczania pakietów, w obecnym modelu rozgrywki nie wpływa to znacząco na odbiór gry i pozwala zachować płynność komunikacji.

6. Obsługa programu

6.1. Budowanie

Projekt można zbudować z katalogu głównego repozytorium za pomocą polecenia:

```
cmake --build build
```

Po poprawnym zbudowaniu w katalogu `build` powinny znajdować się pliki wykonywalne:

- `SwarmServer.exe`,
- `SwarmClient.exe`.

6.2. Uruchamianie

Najpierw należy uruchomić serwer:

```
build\SwarmServer.exe
```

Następnie uruchamiany jest klient:

```
build\SwarmClient.exe
```

Po uruchomieniu klient wyświetla menu, w którym użytkownik podaje adres IP serwera i wybiera klasę bohatera. Po zaakceptowaniu serwer przydziela unikalny identyfikator gracza i rozpoczyna się właściwa rozgrywka.

6.3. Sterowanie

Sterowanie w aktualnej wersji klienta opiera się na standardowym układzie klawiatura plus mysz:

- `W`, `A`, `S`, `D` – poruszanie postacią,
- lewy przycisk myszy – atak podstawowy,
- prawy przycisk myszy – alternatywna umiejętność lub atak dodatkowy, jeśli dana klasa go obsługuje,
- `Shift` – umiejętność ruchowa,
- `E` – slot umiejętności klasowej,
- `Q` – ultimate,
- `R` – przeładowanie broni.

Celowanie odbywa się przy pomocy kursora myszy. Warto zaznaczyć, że nie wszystkie przyciski mają jeszcze pełną funkcjonalność dla każdej klasy, ponieważ część umiejętności pozostaje w trakcie implementacji.

6.4. Wymagane zasoby

Program wymaga obecności katalogu `assets` z plikami:

- `heroes.json`,
- `weapons.json`,
- `enemies.json`.

Brak tych plików uniemożliwia poprawne zainicjowanie rejestrów i może skutkować wyjątkami przy próbie odczytu statystyk.